

A DEVELOPMENT PROPOSAL FOR FORGE

Beyond Local Certificates

Goal-generated invariants, finite-state contracts,
boundary-safe telescoping

Concrete algorithms, mathematical guarantees,
independently replayed Python artifacts, and a Lean integration plan

Prepared for	Vladimir Reshetnikov
Date	15 September 2026
Forge baseline	674521027d968d59f7b83220ed52304a85cb55e2
Implementation	Four Python producers; a separate exact checker
Evidence	317 cases; 313 certificates; 25 rejected invalid mutations
Lean status	No Lean execution in this package; integration remains proposed

This is an additive research proposal, not a replacement for Forge's existing article. Its new implementation is not an installed Lean tactic. A Python acceptance is not a Lean-kernel proof.

Abstract

Forge’s present design already has an extensive proof-planning architecture. The most useful next contribution is therefore not another scheduler, another generalization heuristic, or another statement of the search/checking boundary. This report develops three additional capability areas through four executable producers. First, a goal-generated pullback algorithm constructs a finite vector of polynomial invariants whose entries may change together. Second, ideal closure replaces constant linear dependence with polynomial dependence, resolving nonlinear cases in which the vector-space algorithm necessarily keeps growing. The ideal worker retains original-goal provenance, obtains explicit multiplier certificates, and has an unbounded termination argument for a precisely stated unguarded polynomial fragment. Third, finite-state relational certificates establish equality of fold observations for every finite input word, or produce a shortest distinguishing word. Fourth, a restricted creative-telescoping worker discovers recurrences for binomial-power moments with moving support and explicit endpoint reasoning.

All four producers were executed. A separate standard-library-only checker replays their sparse polynomial identities, finite relations, and concrete counterexamples without importing SymPy. The recorded run contains 178 positive certificates, 135 counterexample certificates, and four resource-bounded unknown outcomes; 240 finite-state results agree with an independent greatest-fixed-point oracle. The report supplies complete mathematical arguments for the certificate rules, concrete integration interfaces, a dependency plan, and adversarial tests. It makes no measured claim against **grind** or any other Lean tactic. The algorithms build on established algebraic invariant analysis, automata equivalence, and creative telescoping; the contribution is their focused adaptation and executable extension of the reviewed Forge design, not a claim to have invented those fields.

Contents

1	The development target and the non-duplication boundary	4
1.1	What should become more capable?	4
1.2	What the repository already contains	4
1.3	Review scope and version discipline	5
1.4	Four meanings of success that must remain separate	5
1.5	Prior work and what is being contributed	6
2	Goal-generated polynomial relation closure	6
2.1	A small problem class with a useful infinite quantifier	6
2.2	The certificate and its complete soundness argument	6
2.3	Discovering the basis from the target	7
2.4	Termination and relative completeness	8
2.5	Worked example: a coupled quadratic relation	8
2.6	Worked example: a sign-changing invariant	9
2.7	A precise obstruction to vector closure	9
3	Goal-generated ideal closure for nonlinear induction	9
3.1	The multiplier certificate	9
3.2	An algorithm that avoids a bilinear synthesis problem	10
3.3	Why the unbounded algorithm terminates	11

3.4	Why a radical ideal is unnecessary	12
3.5	Worked nonlinear separation: repeated squaring	12
3.6	Worked generator discovery beyond one equation	12
3.7	Guards, control flow, and initial assumptions	13
3.8	A further extension: arbitrary polynomial input values	13
4	Finite relational contracts for unbounded folds	14
4.1	From sampled behavior to a finite invariant	14
4.2	The relation certificate	14
4.3	Discovery and shortest counterexamples	14
4.4	An explicit finite-testing failure	15
4.5	The source-to-machine bridge is the real integration work	15
4.6	How to use counterexamples during synthesis	16
4.7	Relational products with the algebraic worker	16
5	Boundary-safe telescoping for binomial-power moments	17
5.1	The new obligation is a sum, not a polynomial antidifference	17
5.2	The exact polynomial identity that search must find	17
5.3	The semantic bridge, including both endpoints	18
5.4	The central-binomial identity, discovered and checked	18
5.5	Regularity is a separate certificate	19
5.6	Selected discovered recurrences	19
5.7	Failure modes and the next genuinely useful generalization	20
6	A concrete Lean and Leant integration plan	20
6.1	Implement the acceptance lemmas before the search adapter	20
6.2	Proposed modules and their exact responsibilities	21
6.3	Reifying an iteration without changing its meaning	21
6.4	Replay through ordinary Lean terms	22
6.5	Finite-state replay and existing Mathlib infrastructure	23
6.6	Telescoping replay and finite sums	23
6.7	Concrete dependencies, without a speculative dependency stack	24
6.8	Selecting a worker from a proof obstruction	24
7	Executed experiments and what they establish	25
7.1	The recorded run	25
7.2	What was generated	25
7.3	The meaningful ablations	26
7.4	Mutation tests and independent arithmetic tests	26
7.5	Symbolic certificates versus numerical diagnostics	26

7.6	Resource settings and one retained failed development run	27
7.7	Timing and its limitations	27
7.8	What the evidence does and does not support	27
8	Implementation priorities and acceptance gates	28
8.1	First milestone: one complete polynomial proof path	28
8.2	Second milestone: the nonlinear ideal path	28
8.3	Third milestone: a universal behavioral assertion	28
8.4	Fourth milestone: summation with checked boundaries	28
8.5	The first fair Lean evaluation	29
8.6	Research directions that remain genuinely open in this design	29
9	Conclusion	29
A	Artifact inventory and reproduction	30
A.1	Files in the archive	30
A.2	Replaying certificates without the producer dependencies	30
A.3	Reproducing discovery and the tests	31
A.4	Building the article	31
B	Wire format and checking obligations	31
B.1	Exact arithmetic	31
B.2	Polynomial closure bundles	32
B.3	Finite-state and telescoping bundles	32
C	Review ledger and provenance limits	32
	References	33

1 The development target and the non-duplication boundary

1.1 What should become more capable?

A useful automation extension changes the class of obligations that can be discharged without a user inventing an intermediate mathematical object. For this report the objects are a *closed family of polynomial relations*, a *finite relational invariant*, and a *telescoping flux*. Each can be difficult to discover and short to check. They also cross a boundary that ordinary local normalization does not cross: they explain all iterations or all summation indices at once.

The proposed development target is not to replace `grind`. It is to give Forge additional ways to manufacture a finite, independently checkable reason for an infinite statement. The local Lean engine remains responsible for the ordinary equalities and logical consequences after the new certificate has been replayed. Nothing here depends on an undocumented modification of its internal congruence state.

There are four named producers:

Producer	Search object	Acceptance object
PRC	Pullback-invariant vector space	Constant-matrix polynomial identities
GI	Pullback-invariant polynomial ideal	Polynomial-multiplier identities
FRC	Reachable product of finite machines	Successor-closed relation or distinguishing word
BFT	First-order binomial telescoper	One exact flux identity and a regularity certificate

PRC means polynomial relation closure; GI means goal-generated ideal closure; FRC means finite relational contract; BFT means binomial flux telescoping. These labels are local package names, not proposed replacements for established terminology.

1.2 What the repository already contains

The reviewed Forge README describes a design rather than an installed tactic and distinguishes original proposal evidence from the merge’s later elaboration of three Lean-core files [1]. The structural section already contains induction generalization, auxiliary-lemma exploration, and exact polynomial recurrence synthesis. Its state-machine certificate initially requires a conserved quantity, $I(F(s)) = I(s)$. It then identifies the stronger condition $I(s) = 0 \Rightarrow I(F(s)) = 0$ and discusses joint polynomial multipliers, warning correctly that simultaneously unknown generators and multipliers create a bilinear problem [2].

Those observations are the starting point, not new contributions claimed here. The new algebraic work gives a concrete algorithm that *constructs the generators from the goal*, closes them, and only then reconstructs multipliers. This supplies an executable answer to that warning without pretending that a bilinear system is a linear one.

The merged nonlinear and external-engine sections already cover several rich certificate languages and integration techniques [3, 4]. This report does not redescribe their sum-of-squares, Bernstein, Sturm, SAT/theory, equality-saturation, or external-prover designs. Likewise, the project’s Leant/Djex review already discusses the verification boundary, exact identity, evidence permissions, and negative-result gates [5]. Those disciplines remain requirements for the eventual integration; naming them again would not make Forge more powerful.

Area	Already present in the reviewed design	Increment delivered here
Polynomial invariants	Conserved identities; additive recurrence synthesis; joint-multiplier condition discussed	Goal-orbit basis discovery, finite-dimensional closure theorem, original-word counterexamples
Nonlinear induction	General implication invariant worker proposed	Ascending-ideal discovery with explicit polynomial multipliers and bounded executable reconstruction
Higher-order behavior	Type-directed recursor/fold synthesis and behavioral checks	A finite-machine universal-contract certificate, including an explicit source-to-machine bridge obligation
Summation	Polynomial antidifferences for additive recurrences	Moving-support binomial-power summation with a searched flux and endpoint-safe recurrence proof

1.3 Review scope and version discipline

The baseline is Forge commit `674521027d968d59f7b83220ed52304a85cb55e2`. The review used GitHub connector reads of the README, substantial ranges of the merged structural, nonlinear, and external-engine sections, and the Leant/Djex integration review. The repository tree was inspected through the connector, but its large response was truncated; no complete local checkout was obtained. In particular, this is *not* a line-by-line audit of all nine frozen submissions. Empty code-search results were not treated as proof of absence.

The non-duplication claim is correspondingly specific: the algorithmic details and artifacts in this report extend the reviewed merged design, rather than republishing its existing material. Broad statements such as “invariant synthesis” or “finite-state reasoning” are not claimed as globally new ideas. The audit ledger names the exact files and ranges read, so a maintainer can compare against a later repository revision without relying on a vague assertion that everything was reviewed.

Leant was observed at commit `6bf05ad78c46`; Djex at commit `e8778f4ebd63`. Full identifiers are recorded in the bibliography and audit ledger. Their READMEs and Leant’s behavioral-synthesis documentation were used for the integration boundary [6, 8, 7]. No claim is made to have compiled or exhaustively audited either project.

1.4 Four meanings of success that must remain separate

The report distinguishes a mathematical theorem about a certificate rule, successful execution of a Python search, successful independent Python replay, and successful Lean-kernel replay. The first three are provided. The fourth is not: there was no Lean executable in this environment. The Lean source specimen is therefore marked `NOT_RUN`, and no installed `forge` syntax is advertised.

A returned positive certificate means the mathematical input encoded in the bundle satisfies the certificate rule. It does not establish that an arbitrary Lean expression was translated correctly into that input. A returned counterexample is a checked execution word for the encoded transition system, or a distinguishing word for the encoded machines. A telescoper certificate proves a recurrence through the mathematical bridge developed below; it does not by itself prove an unrelated guessed closed form. These are substantive scope restrictions, not cosmetic labels.

1.5 Prior work and what is being contributed

Polynomial weakest-precondition analysis and ideal stabilization have established antecedents; Müller-Olm and Seidl give an algebraic analysis of polynomial program invariants [9]. Automata equivalence has a substantial classical and coalgebraic literature [10]. Creative telescoping is a mature subject, for which Koutschan gives a useful account [11]. The proofs below specialize these ideas to small acceptance languages suitable for Forge and make all relevant assumptions explicit.

The contribution is therefore engineering and mathematical adaptation: a demand-generated certificate interface, provenance-preserving search, a stronger nonlinear route that avoids jointly unknown multipliers, a universal behavioral worker that fits Leant’s exact-candidate boundary, a moving-boundary summation schema, and an executable evidence package. The most important claim is that these additions can be implemented and audited without trusting their search algorithms. The accompanying code demonstrates that claim at the Python level.

2 Goal-generated polynomial relation closure

2.1 A small problem class with a useful infinite quantifier

Let A be a finite nonempty alphabet. For each $a \in A$, let $F_a : \mathbb{Q}^d \rightarrow \mathbb{Q}^d$ be a total polynomial map with rational coefficients. Fix $s_0 \in \mathbb{Q}^d$ and target polynomials $g_1, \dots, g_t \in \mathbb{Q}[X_1, \dots, X_d]$. A word $w = a_1 \cdots a_\ell$ acts in execution order:

$$F_w = F_{a_\ell} \circ \cdots \circ F_{a_1}, \quad F_\varepsilon = \text{id}.$$

The requested assertion is

$$\forall w \in A^*, \quad \forall j \leq t, \quad g_j(F_w(s_0)) = 0. \quad (1)$$

One symbol describes a recurrence iterated an arbitrary number of times. Several symbols describe every interleaving of a fixed collection of updates. Running two implementations on the same word is represented by a product state, so the targets may express relations between them.

This input class does not include guards, partial transitions, truncated subtraction, integer division, or arbitrary recursive data. A frontend may reduce a larger problem to this class only with a proved bridge. In particular, a negative execution of an unguarded abstraction need not be a feasible execution of a guarded source program.

Definition 2.1 (Pullback). For a polynomial q , define $F_a^*q = q \circ F_a$. This is substitution of the coordinate polynomials of F_a for the variables of q , performed *simultaneously*.

A pullback asks which property before a step would imply the desired property after the step. Unlike forward trajectory sampling, it preserves a symbolic statement about every state.

2.2 The certificate and its complete soundness argument

Search for a column $B = (b_1, \dots, b_r)^T$ of rational polynomials, matrices $H_a \in \mathbb{Q}^{r \times r}$, and a matrix $C \in \mathbb{Q}^{t \times r}$ satisfying

$$B(s_0) = 0, \quad (2)$$

$$B(F_a(s)) = H_a B(s) \quad (a \in A), \quad (3)$$

$$(g_1(s), \dots, g_t(s))^T = C B(s). \quad (4)$$

The second and third lines are polynomial identities, not equalities tested on a finite grid. Individual b_i may grow, alternate sign, or mix with other entries. Only their common zero set must be preserved.

Theorem 2.2 (Vector closure certificate). *If (2)–(4) hold, then (1) holds.*

Proof. Induct on execution length. The empty execution has $B(s_0) = 0$. If $B(s) = 0$ and the next symbol is a , then $B(F_a(s)) = H_a 0 = 0$. Hence every reachable state has $B = 0$. Multiplication by C gives every target equality. $\square$

The checker does not need to verify that the b_i are independent, minimal, or actually discovered by the advertised algorithm. Those properties affect search cost, not soundness. It checks the original targets, every original transition, the initial evaluations, and the displayed identities. The search-origin annotations are deliberately irrelevant to positive acceptance.

2.3 Discovering the basis from the target

Begin with the targets in a FIFO worklist and an empty basis. For each polynomial q removed from the worklist, test whether it belongs to the rational span of the current basis. If so, discard it. Otherwise append it and enqueue its pullbacks along every transition. Exact row reduction implements span membership.

```
B := []; work := [(g_j, target=j, word=[]) for j]
while work is nonempty:
  (q, j, w) := pop_front(work)
  if polynomial or work budgets exceeded: return UNKNOWN
  if q(s0) != 0: return checked_word_counterexample(j, w)
  if q is in span_Q(B): continue
  if basis budget exhausted: return UNKNOWN
  append q to B
  for a in alphabet:
    enqueue(q composed with F_a, j, [a] ++ w)
reconstruct H_a and C over exact rationals
return certificate only after independent replay
```

This is not enumeration of all degree- D invariant templates. Only the closure generated by the goal is explored. It resembles a block Krylov construction, but the operators are symbolic pullbacks and there may be several of them. The implementation recomputes exact coordinate systems for clarity; an incremental echelon basis is a straightforward performance improvement.

Why a discarded polynomial needs no descendants. If $q = \sum_i c_i b_i$, then $q \circ F_a = \sum_i c_i (b_i \circ F_a)$. Once the basis descendants are accounted for, the descendants of q add no new direction. This is the mathematical justification for the worklist pruning; it does not depend on a heuristic estimate of relevance.

Why word order is easy to get wrong. Suppose $q = g_j \circ F_w$. Then

$$q \circ F_a = g_j \circ F_w \circ F_a = g_j \circ F_{aw}.$$

Pulling back along a prepends a to the execution word. The implementation stores this exact provenance. A test with updates $F_0(x, y) = (1, y)$ and $F_1(x, y) = (x, x)$, initial state $(0, 0)$, and target $y = 0$ finds $[0, 1]$. Its reverse $[1, 0]$ is not a counterexample, and the independent checker rejects that mutation.

2.4 Termination and relative completeness

Let

$$V = \text{span}_{\mathbb{Q}}\{g_j \circ F_w : j \leq t, w \in A^*\}.$$

Theorem 2.3 (Finite orbit-span guarantee). *If $\dim V = r < \infty$, exact uncapped vector closure terminates. If (1) is true, it returns a positive certificate. If (1) is false, it returns a concrete counterexample. At most r basis insertions and $|A|r$ newly generated pullbacks are needed.*

Proof. Every inserted polynomial lies in V and is independent of the previous ones, so there are at most r insertions. Each insertion enqueues finitely many descendants. The queue is therefore finite. At exhaustion, the resulting span contains all targets and is invariant under every pullback, because each basis pullback was processed. It follows by induction on word length that it equals V .

If some processed $q = g_j \circ F_w$ has $q(s_0) \neq 0$, its stored word refutes the assertion. If no such polynomial exists, every basis member vanishes at s_0 , giving the initial certificate condition. Exact coordinate reconstruction gives the other conditions. Were there an unreported counterexample after closure, its target pullback would be a linear combination of basis polynomials, all zero at s_0 , a contradiction. $\square$

Corollary 2.4 (Affine transition fragment). *For affine F_a and targets of total degree at most D , vector closure terminates in a space of dimension at most $\binom{d+D}{D}$.*

Proof. Affine substitution preserves the degree bound. The monomials of total degree at most D form a vector-space basis of the stated size. $\square$

The dimension bound does not make exact arithmetic cheap. Rational numerators and denominators can grow, and a naive implementation repeatedly builds dense matrices over all current monomials. The prototype has explicit degree, term, basis, and pullback ceilings. A ceiling gives UNKNOWN, even on a mathematically complete fragment. The counterexample is not advertised as shortest in this algebraic worker: linear pruning and multiple targets complicate that additional promise. Only the finite-machine BFS below promises shortest distinguishing words.

2.5 Worked example: a coupled quadratic relation

Consider synchronized Fibonacci-style updates

$$F(x, y, u, v) = (y, x + y, v, u + v), \quad s_0 = (0, 1, 0, 1),$$

and the target

$$b_0 = x^2 + y^2 - u^2 - v^2.$$

The target is not conserved as a polynomial. Its pullback is

$$b_1 = x^2 + 2xy + 2y^2 - u^2 - 2uv - 2v^2.$$

A second pullback satisfies

$$b_1 \circ F = -b_0 + 3b_1.$$

Thus the search discovers

$$B = \begin{pmatrix} b_0 \\ b_1 \end{pmatrix}, \quad H = \begin{pmatrix} 0 & 1 \\ -1 & 3 \end{pmatrix}, \quad C = \begin{pmatrix} 1 & 0 \end{pmatrix}. \quad (5)$$

Both entries vanish initially, so the joint relation is inductive. There is no need to guess b_1 in advance, enumerate quadratic invariants, or ask a nonlinear solver to invent a matrix and basis simultaneously. The particular synchronized example has an easier human proof by equality of the two state pairs. Its purpose is not to make a benchmark look difficult. It isolates the discovery mechanism and exposes precisely why requiring $b_0 \circ F = b_0$ would be too restrictive. The generated affine corpus varies the two updates and the quadratic observation rather than claiming this single example measures realistic Lean difficulty.

2.6 Worked example: a sign-changing invariant

With

$$F(x, y, t) = (y, x + y, -t), \quad s_0 = (0, 1, 1), \quad b = y^2 - xy - x^2 - t,$$

exact expansion gives $b \circ F = -b$. The worker returns the one-by-one matrix $H = (-1)$. The conserved-quantity condition $b \circ F = b$ fails, but the zero set remains invariant. This gives a Cassini-shaped relation while representing the alternating sign by an extra state coordinate, rather than incorrectly reifying $(-1)^n$ as a polynomial in n .

2.7 A precise obstruction to vector closure

For $F(x, y) = (x^2, y^2)$ and $g = x - y$, the successive pullbacks are $x^{2^j} - y^{2^j}$. They are linearly independent over $\mathbb{Q}$: in a finite combination, the greatest exponent occurs in exactly one summand and cannot cancel. Hence V is infinite-dimensional. An arbitrarily generous finite vector-space budget cannot establish closure this way.

Nevertheless, the invariant is elementary:

$$(x - y) \circ F = (x + y)(x - y).$$

The obstacle is the restriction to *constant* coefficients. Allowing a polynomial multiplier changes the mathematics of closure and motivates the next worker. This is an algorithmic routing criterion, not merely a request to increase fuel.

3 Goal-generated ideal closure for nonlinear induction

3.1 The multiplier certificate

Keep the transition-system input of Section 2, but now allow the entries of H_a and C to be polynomials in the state. The certificate conditions become

$$b_i(s_0) = 0 \quad (1 \leq i \leq r), \quad (6)$$

$$b_i(F_a(X)) = \sum_{j=1}^r h_{aij}(X) b_j(X), \quad (7)$$

$$g_\ell(X) = \sum_{j=1}^r c_{\ell j}(X) b_j(X). \quad (8)$$

No multiplier has a sign condition. These are equality proofs, not positivity certificates. The acceptance language is deliberately just polynomial addition, multiplication, substitution, and rational equality.

Theorem 3.1 (Ideal closure certificate). *Conditions (6)–(8) imply (1).*

Proof. Assume every $b_j(s)$ is zero. Evaluating (7) at s gives $b_i(F_a(s)) = 0$, because every summand contains a zero factor. The initial condition and induction show that every basis polynomial vanishes at every reachable state. Evaluation of (8) proves the target equalities. $\square$

This proof does not use Gröbner bases, ideal membership algorithms, algebraic closure of the coefficient field, or a completeness theorem. Those enter search only. It works over any commutative ring in which the displayed coefficient and substitution identities are valid; the implemented rational coefficient representation imposes its own field-embedding obligations on a Lean frontend.

3.2 An algorithm that avoids a bilinear synthesis problem

Write $R = \mathbb{Q}[X_1, \dots, X_d]$. The ideal generated by $b_1, \dots, b_r$ is

$$\langle b_1, \dots, b_r \rangle = \left\{ \sum_j h_j b_j : h_j \in R \right\}.$$

Instead of choosing an unknown template for every b_j and every h_j , construct the b_j from actual pullbacks of the goal. When a candidate q is not in the current ideal, append q itself as a new generator. Membership is decided during search by an untrusted exact Gröbner computation.

```
B := []; J := zero ideal; work := original targets with origins
while work is nonempty:
  (q, goal, word) := pop_front(work)
  enforce resource limits
  if q(s0) != 0: return checked concrete word counterexample
  if q belongs to J: continue
  append the original polynomial q to B
  recompute a search-only Groebner basis for J = ideal(B)
  enqueue q composed with each transition, retaining word order
freeze B
for each required pullback and each target:
  reconstruct multipliers relative to the frozen B
  if reconstruction exceeds its separate budget: return UNKNOWN
return the explicit identities only after independent replay
```

The Gröbner basis is *not* substituted for the retained generators. It is an index for membership queries. Retaining raw pullbacks makes a negative result interpretable as an execution of the original target and avoids making positive replay depend on a proof of the whole Gröbner algorithm.

Why fixed-generator reconstruction is linear. Once B is frozen, choose a multiplier degree e and write

$$h_j(X) = \sum_{|\alpha| \leq e} u_{j\alpha} X^\alpha.$$

The equation $q = \sum_j h_j b_j$ is linear in the unknown rational numbers $u_{j\alpha}$. Every coefficient of each b_j is already fixed. The prototype first tries ordinary multivariate division by the raw generator list, then solves the exact coefficient system for increasing e within the cap. An unsuccessful solve at one degree is not evidence that ideal membership is false.

A production producer could instead track transformation vectors while computing a Gröbner basis. If $G_k = \sum_j T_{kj} b_j$ and a reduction gives $q = \sum_k v_k G_k$, then $h_j = \sum_k v_k T_{kj}$. The same final identity checker suffices. The current package implements the fixed-generator linear reconstruction, not that faster provenance-carrying Gröbner backend.

3.3 Why the unbounded algorithm terminates

Define the pullback-generated ideal

$$J_* = \langle g_\ell \circ F_w : \ell \leq t, w \in A^* \rangle.$$

Every insertion strictly increases the current ideal. Unlike the vector space of all pullbacks, R is Noetherian: every ascending chain of ideals stabilizes.

Theorem 3.2 (Unbounded ideal discovery). *For finitely many total rational polynomial transitions, finitely many polynomial targets, and a fixed rational initial state, the uncapped discovery algorithm terminates. If multiplier reconstruction is allowed increasing finite degree bounds without a global cutoff, it returns either a certificate proving (1) or a concrete execution refuting it.*

Proof. Let $J_0 \subsetneq J_1 \subsetneq \dots$ be the ideals after successful insertions. The Hilbert basis theorem forbids infinitely many strict inclusions. Each insertion creates only $|A|$ new work items, so after finitely many insertions the worklist also becomes empty.

Every retained generator is an actual target pullback, so its nonzero evaluation at the initial state yields a concrete counterexample. Suppose no such counterexample was returned. Every retained generator vanishes at s_0 .

At exhaustion the ideal J contains every target and every pullback of each retained generator. Since substitution is a ring homomorphism,

$$F_a^* \left(\sum_j h_j b_j \right) = \sum_j (F_a^* h_j) (F_a^* b_j) \in J.$$

Thus J is closed under every pullback and contains all of J_* . Conversely every inserted generator belongs to J_* , so $J = J_*$. The required step and target identities therefore have polynomial multipliers. Each such representation has some finite degree; increasing-degree coefficient reconstruction eventually finds one. The initial, step, and target conditions then give a positive certificate.

Finally, if a counterexample w existed after positive closure, $g_\ell \circ F_w \in J_*$ would evaluate to zero at s_0 , since every generator does. This contradicts its definition as a counterexample. $\square$

Remark 3.3 (Precisely what was decided). This is a decision result for equality observations on the closed, unguarded polynomial system just defined. It is not a decision result for arbitrary program verification, real inequalities, unrestricted guards, termination, general Lean goals, or the existence of a term inhabiting a dependent type. The executed implementation is resource-bounded and therefore does not implement an unconditional decision interface.

The Noetherian proof is a termination argument, not a practical degree or runtime bound. A single Gröbner calculation may dominate runtime and memory. The current synchronous SymPy calls do not provide fine-grained cancellation. Production deployment must run the producer under an external process deadline and memory limit; a timeout is UNKNOWN. This caveat is not removed by the existence of a mathematical termination theorem.

3.4 Why a radical ideal is unnecessary

It is tempting to replace J by its radical because only zero sets matter. That can shrink the generating set, but it creates new proof obligations. A radical-membership claim $q^m \in J$ only yields $q = 0$ in a setting with appropriate reducedness or integral-domain properties. The direct multiplier certificate makes no such inference.

For this task ordinary ideals already yield the termination theorem. The package deliberately avoids radical computation. A later producer may use radical information to propose simpler generators, but it must add a replay rule that proves the relevant root-of-zero implication in the source coefficient domain. This is a concrete place where a search optimization would otherwise silently strengthen the logic of the checker.

3.5 Worked nonlinear separation: repeated squaring

Let $F(x, y) = (x^2, y^2)$, $s_0 = (2, 2)$, and $g = x - y$. The vector worker reaches its degree ceiling after seeing degree 16, having retained four independent orbit polynomials. The ideal worker retains only $b_1 = x - y$ and supplies

$$b_1(F(x, y)) = (x + y)b_1(x, y), \quad g = b_1.$$

The independent checker expands both sides and checks $b_1(2, 2) = 0$. The fact that values become enormous under iteration is irrelevant to positive checking: no long trajectory is evaluated.

The generated nonlinear corpus uses two synchronized scalar maps, including powers two through four, linear terms, and constants. It is intentionally a small structural family with known equivalence. Its value is to test multiplier generation across different symbolic updates; it is not an unbiased measure of all nonlinear verification problems.

3.6 Worked generator discovery beyond one equation

Take $F(x, y) = (y, x + y)$, $s_0 = (0, 0)$, and the target $b_0 = xy$. The first pullback is

$$b_1 = xy + y^2,$$

which does not belong to $\langle xy \rangle$. Its pullback is

$$b_2 = x^2 + 3xy + 2y^2,$$

which adds another generator. These three span the quadratic monomials as a vector space and therefore generate the ideal $(x, y)^2$. The next pullback satisfies

$$b_2 \circ F = -b_0 + 2b_1 + 2b_2.$$

Consequently the generated joint zero condition is inductive. This illustrates the distinction between simply verifying a user-supplied ideal invariant and discovering the stronger invariant needed by the goal.

The initial state in this example remains fixed, making the theorem itself easy. That does not invalidate the generator-growth test, but it does prevent the case from supporting a claim of superiority to existing Lean automation. The package keeps that distinction explicit.

3.7 Guards, control flow, and initial assumptions

For guarded transitions, an unconditional multiplier identity is still a sufficient certificate: it proves preservation even when the guard is discarded. The proof may become harder or impossible after that abstraction. A counterexample word in the abstraction must not refute the source statement until every guard is checked on the resulting concrete states.

A practical extension for finite control-flow graphs uses one generator family B_v per program point. An edge $u \xrightarrow{F} v$ requires

$$B_v(F(X)) = H_{uv}(X)B_u(X).$$

Targets seed their program points, and work propagates backward across incoming edges. A finite tuple of ideal lattices over Noetherian rings still admits the same ascending-chain argument. Initial evaluation is needed only at entry points. This extension is specified here, not implemented in the supplied single-location worker.

For an initial state parameterized polynomially by z , replace the scalar initial checks with identities $b_i(s_0(z)) = 0$. That proves the result for every rational parameter value. For an initial *assumption* ideal $K = \langle k_1, \dots, k_m \rangle$, a sufficient initial certificate is $b_i = \sum_j v_{ij}k_j$. This need not be complete for every semantic initial zero set unless further algebraic reasoning is introduced; the implementation must not claim completeness merely from a sufficient ideal-membership test.

3.8 A further extension: arbitrary polynomial input values

Finite alphabets are useful, but many folds consume integers or rationals. Suppose a transition is polynomial in both state X and an arbitrary input Z . Expand

$$b_i(F(X, Z)) = \sum_{\alpha} q_{i\alpha}(X)Z^{\alpha}.$$

A sufficient closure certificate expresses every coefficient $q_{i\alpha}$ in the state ideal. Equivalently, supply multipliers $h_{ij}(X, Z)$ with

$$b_i(F(X, Z)) = \sum_j h_{ij}(X, Z)b_j(X).$$

This establishes preservation for *all* input values, and then induction establishes a contract for lists of arbitrary length and arbitrary rational entries.

For discovery, enqueue the finitely many coefficient polynomials of each pullback, rather than treating infinitely many input values as separate transition symbols. Each insertion still strictly increases a state ideal, so Noetherianity gives a positive closure route. However, the raw-word counterexample mechanism must change: a coefficient polynomial is not itself the observation along a concrete input word. One must retain the parameterized word and reconstruct actual rational inputs, or restrict this extension to positive evidence. The package does not implement this extension and does not reuse the finite-alphabet negative certificate unmodified.

This coefficientwise extension is particularly relevant to Forge’s relationship with Leant. A universal assertion about a polynomial accumulator can be discharged through a finite symbolic induction invariant even when the input element type is infinite. Type-directed synthesis can propose the program; this worker can prove a scoped behavioral contract, without assuming that type correctness implies the intended behavior.

4 Finite relational contracts for unbounded folds

4.1 From sampled behavior to a finite invariant

Leant’s documented behavioral entrance checks a proposition for the exact type-correct candidate, using bounded attempts with `decide` and `simp`; it distinguishes passing, falsification, and inconclusive outcomes [7]. The proposed addition is a worker for a universal proposition when the candidate and the reference have finite-state semantics. It is not another sample filter.

A deterministic Moore machine is a tuple

$$M = (Q, A, \delta, q_0, o),$$

where Q and A are finite sets, $\delta : Q \times A \rightarrow Q$ is total, and $o : Q \rightarrow O$ is an observation function. Reading a word updates the state with a left fold. The implemented output set consists of exact integers; equality is the only operation needed on outputs.

For two machines sharing A , the contract is

$$\forall w \in A^*, \quad o_L(\text{run}_L(q_{L0}, w)) = o_R(\text{run}_R(q_{R0}, w)). \quad (9)$$

This covers recognizers, finite protocol states, modular counters represented by finite tables, and finite-control folds. It does not directly cover arbitrary list transducers returning unbounded lists, general Church encodings, or infinite-state accumulators.

4.2 The relation certificate

A certificate is a finite relation $\mathcal{R} \subseteq Q_L \times Q_R$ satisfying

$$(q_{L0}, q_{R0}) \in \mathcal{R}, \quad (10)$$

$$(p, q) \in \mathcal{R} \implies o_L(p) = o_R(q), \quad (11)$$

$$(p, q) \in \mathcal{R} \implies (\delta_L(p, a), \delta_R(q, a)) \in \mathcal{R} \quad (a \in A). \quad (12)$$

The checker verifies all three conditions against the independently supplied transition and output tables. It need not check that the relation is minimal, that every listed pair is reachable, or that a particular traversal produced it.

Theorem 4.1 (Finite relational contract). *A relation satisfying (10)–(12) proves (9).*

Proof. The initial pair is related. Every common input symbol sends a related pair to another related pair. Induction on the input word therefore relates the final states. Their observations agree by (11). $\square$

This is a finite certificate for infinitely many finite words. The induction theorem is small; the finite table checks do not themselves enumerate those words.

4.3 Discovery and shortest counterexamples

The producer performs BFS on the product transition system, starting from the initial pair. A visited pair with unequal outputs immediately yields a distinguishing word by following parent pointers. If the queue empties without a mismatch, the visited relation is a positive certificate.

```

queue := [initial_pair]; parent[initial_pair] := none
while queue nonempty:
  pair := pop_front(queue)
  if outputs disagree:
    return independently checked parent-path word
  for symbol in alphabet:
    successor := product_step(pair, symbol)
    if successor is new:
      if pair budget exhausted: return UNKNOWN
      record parent and enqueue successor
return independently checked visited relation

```

Proposition 4.2 (Finite-state decision and cost). *Without a pair ceiling, the algorithm terminates and decides (9). A returned distinguishing word has minimum length. The traversal uses $O(|A| |Q_L| |Q_R|)$ transition inspections and $O(|Q_L| |Q_R|)$ stored pairs in the worst case.*

Proof. Each product pair is visited at most once. BFS processes pairs by nondecreasing distance from the root, so the first mismatch has shortest distance. If no mismatch is reachable, the visited set is closed under every transition and satisfies the relation certificate. Conversely any output mismatch reached by a word falsifies the contract. $\square$

Faster equivalence algorithms are known [10]; the prototype does not claim their complexity. Product BFS was chosen because it provides a transparent relation and a shortest witness with almost no extra machinery. A more sophisticated producer can retain the same checker.

4.4 An explicit finite-testing failure

Construct a unary machine with states $0, 1, \dots, 17$, transitions $i \mapsto \min(i + 1, 17)$, and output 1 only at state 17. Compare it with a one-state machine constantly outputting 0. The observations agree at every length from zero through eight, so the selected finite sample suite passes. BFS finds the first difference at length 17.

The experiment also runs with a pair budget of eight. That run returns UNKNOWN, not equivalence. Thus the same example exercises both errors that a behavioral integration must avoid: promoting successful samples to a universal result, and promoting an incomplete exploration to a universal result. The complete run returns an actual word, not merely a message that a search branch failed.

4.5 The source-to-machine bridge is the real integration work

A certificate about a table is not automatically a certificate about a Lean function. Let the source state type be S , its step function be $f : S \rightarrow A \rightarrow S$, and let $\eta : S \rightarrow Q$ map source states to machine states. To transfer a finite-state contract, require

$$\eta(s_{\text{init}}) = q_0, \tag{13}$$

$$\eta(f(s, a)) = \delta(\eta(s), a), \tag{14}$$

$$o_{\text{source}}(s) = o(\eta(s)). \tag{15}$$

A restricted source invariant may be threaded through these conditions when the equalities are not valid for every source state. The corresponding induction must carry that invariant explicitly.

Theorem 4.3 (Fold transport). *If (14) holds, then*

$$\eta(\text{run}_f(s, w)) = \text{run}_\delta(\eta(s), w)$$

for every finite word w .

Proof. Induct on w . The empty case is reflexivity. In the constructor case, apply the induction hypothesis from $f(s, a)$, then rewrite $\eta(f(s, a))$ using the commuting-step equality. $\square$

The source bridge should be produced from a *restricted, recognized fold syntax*, not inferred from a suggestive pretty-printed type. The first implementation should accept explicit finite state types, exact constructor cases, and a visibly structural fold. It can then generate the finite table by evaluation and prove each source-step equation by reduction or a short local proof. Defunctionalization of higher-order closures is a later extension: every closure constructor and environment component needs a simulation rule.

Djex’s source-owned term evidence and Leant’s exact-candidate verification are useful at this entrance [8, 6]. They identify which candidate is being analyzed. They do not, by themselves, establish the commuting square. Universe choices, dependent fields, and dictionary-carried data must survive until the source bridge is proved. A Church-like polymorphic type alone is not authorization to replace the term by a finite automaton or to assume a parametricity theorem.

4.6 How to use counterexamples during synthesis

A distinguishing word can reject the *particular* candidate-reference pair after its source bridge is validated and the candidate is evaluated on that word. It can also enrich a reusable pool of behavioral tests for later candidates. A replay must always bind the test to the exact new candidate before rejecting it.

No finite set of such rejected candidates proves that the requested type has no inhabitant satisfying the contract. The universal positive result for one candidate and the universal negative result for a synthesis grammar are different problems. This worker provides the former, and a concrete behavioral falsification of individual candidates. It does not extend Djex’s non-inhabitation authority by implication.

4.7 Relational products with the algebraic worker

Many programs combine finite control with rational data. A natural next worker stores a polynomial generator family for each reachable control-state pair. Each product transition requires a polynomial multiplier certificate between the appropriate families. This combines the proof structure of (12) with (7).

A simple first version handles unconditional polynomial updates on a finite control graph; a guard-dependent version must retain proof obligations for guards. The prototype does not implement this combined worker. Nevertheless, the formats already align: finite relation edges identify which transition identity must be checked, and the polynomial certificate proves that identity. The resulting proof can still be one induction over the input word rather than separate, mutually trusting analyses.

5 Boundary-safe telescoping for binomial-power moments

5.1 The new obligation is a sum, not a polynomial antidifference

The existing structural article handles additive polynomial recurrences by searching for a polynomial antidifference. Here the summand itself depends on the outer parameter and has moving support. Fix an integer $m \geq 1$ and a rational polynomial $w(K)$. Define

$$S(n) = \sum_{k=0}^n w(k) \binom{n}{k}^m, \quad n \in \mathbb{N}. \quad (16)$$

The worker seeks a first-order recurrence

$$a(n)S(n+1) = b(n)S(n) \quad (17)$$

with rational polynomials a, b and $a \neq 0$. This is a restricted creative-telescoping problem, not a general Zeilberger implementation. The general methodology is established [11]; the deliverable here is a small certificate schema whose support boundaries can be proved once and reused.

The moving upper endpoint matters. A formal identity valid only when $0 < k < n$ cannot be summed as though it held at $k = 0$ and $k = n + 1$. Dividing by a binomial coefficient or by $n + 1 - k$ is particularly dangerous at the endpoint. The certificate below is designed to avoid those divisions.

5.2 The exact polynomial identity that search must find

Use indeterminates N, K and set $D = N + 1 - K$. Search for $a(N), b(N)$ and $U(N, K)$ satisfying

$$[a(N)(N+1)^m - b(N)D^m]w(K) = D^m U(N, K+1) - K^m U(N, K). \quad (18)$$

Everything in (18) is polynomial. The checker receives m and w from the original problem, parses the proposed a, b, U , substitutes $K + 1$ exactly, expands independently, and compares coefficients. The producer cannot replace the requested weight or power in the certificate.

Search as exact linear algebra. For bounds d, e , use

$$a(N) = \sum_{i=0}^d a_i N^i, \quad b(N) = \sum_{i=0}^d b_i N^i, \quad U(N, K) = \sum_{i+j \leq e} u_{ij} N^i K^j.$$

Substitution into (18) gives a homogeneous rational linear system. The prototype computes its nullspace, examines basis vectors with nonzero a , clears rational denominators, and performs exact replay. The number of unknown coefficients is

$$2(d+1) + \frac{(e+1)(e+2)}{2}.$$

The search raises d and e within configured bounds. It does not use samples of $S(n)$ to manufacture a recurrence, and it does not trust a symbolic summation function returning an answer without a flux.

The candidate policy is intentionally modest: it inspects nullspace basis vectors and uses a bounded sufficient test for eventual nonvanishing of a . A failure is not a proof that no first-order recurrence exists, and it is not even a general infeasibility certificate for all possible bounded recurrence/regularity combinations. A different nullspace combination, regularity proof, denominator schema, or recurrence order might succeed.

5.3 The semantic bridge, including both endpoints

Interpret binomial coefficients as zero above their upper index. For $n \geq 0$, define

$$G(n, k) = \frac{k^m U(n, k)}{(n+1)^m} \binom{n+1}{k}^m. \quad (19)$$

The only denominator is $(n+1)^m$, which is nonzero. For $0 \leq k \leq n+1$, the scaled binomial identities give

$$(n+1) \binom{n}{k} = (n+1-k) \binom{n+1}{k}, \quad (20)$$

$$(k+1) \binom{n+1}{k+1} = (n+1-k) \binom{n+1}{k}. \quad (21)$$

Both are valid at $k = n+1$: each side then vanishes where required. They can be established without canceling a possibly zero binomial coefficient. Mathlib's pinned binomial module supplies the inductive definition, support-zero lemmas, and scaled identities from which the bridge can be built [15].

Theorem 5.1 (Binomial flux certificate). *If (18) holds as a polynomial identity and $m \geq 1$, the sum (16) satisfies (17) for every $n \in \mathbb{N}$.*

Proof. Fix $n \geq 0$ and $0 \leq k \leq n+1$. Multiply (18), evaluated at (n, k) , by $\binom{n+1}{k}^m / (n+1)^m$. Using (20), its left side becomes

$$a(n)w(k) \binom{n+1}{k}^m - b(n)w(k) \binom{n}{k}^m.$$

Using (21), its right side becomes $G(n, k+1) - G(n, k)$. Thus the local identity is valid on the entire summation range, including its endpoints.

Sum from $k = 0$ to $k = n+1$. The left side is $a(n)S(n+1) - b(n)S(n)$, because the additional $k = n+1$ term in the second sum is zero. The right side telescopes to $G(n, n+2) - G(n, 0)$. The upper flux vanishes because $\binom{n+1}{n+2} = 0$; the lower flux vanishes because $0^m = 0$ for $m \geq 1$. Therefore the difference is zero. $\square$

The Python checker verifies the universal *polynomial* identity and the scalar side conditions. The theorem above is the mathematical bridge from that identity to the sum. This bridge has not been compiled in Lean in the supplied package. It must be proved in Lean before the worker can close an original Lean summation goal.

5.4 The central-binomial identity, discovered and checked

For $m = 2$ and $w = 1$, the producer finds

$$a(N) = N + 1, \quad b(N) = 4N + 2, \quad U(N, K) = 2K - 3N - 3.$$

The checker verifies

$$\begin{aligned} & (N+1)^3 - (4N+2)(N+1-K)^2 \\ &= (N+1-K)^2(2K-3N-1) - K^2(2K-3N-3). \end{aligned}$$

Consequently

$$(n+1) \sum_{k=0}^{n+1} \binom{n+1}{k}^2 = (4n+2) \sum_{k=0}^n \binom{n}{k}^2.$$

The sequence $T(n) = \binom{2n}{n}$ satisfies the same recurrence. For example, applying the factorial formula in its valid range gives

$$\frac{T(n+1)}{T(n)} = \frac{(2n+2)(2n+1)}{(n+1)^2} = \frac{4n+2}{n+1},$$

where the factorials are positive and $n+1$ is nonzero. Both sequences start at one. Induction therefore proves

$$\boxed{\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}}. \quad (22)$$

This identity is classical; it is not a new theorem claimed by the report. The computational result is that the supplied restricted search finds the local certificate without receiving this recurrence or flux as input. The package stores the recurrence certificate, not a machine-checked proof of the factorial argument identifying the closed form.

5.5 Regularity is a separate certificate

A polynomial recurrence can hold at an index where $a(n) = 0$, yet fail to determine $S(n+1)$. The prototype stores a nonnegative integer N_0 and a sign $\sigma \in \{-1, 1\}$ such that

$$\sigma a(N_0 + T) = c_0 + c_1 T + \cdots + c_d T^d, \quad c_0 > 0, \quad c_i \geq 0. \quad (23)$$

Then $a(n) \neq 0$ for every integer $n \geq N_0$. This is a sufficient positivity proof, not a root-isolation decision procedure. The search tries shifts from zero through twenty; the checker validates the supplied shift without trusting the search.

Proposition 5.2 (Using a recurrence to identify a sequence). *Suppose S and T satisfy (17), and $a(n) \neq 0$ for $n \geq N_0$. If $S(N_0) = T(N_0)$, then $S(n) = T(n)$ for every $n \geq N_0$. To conclude equality for all natural indices, also verify every index below N_0 .*

Proof. If equality holds at $n \geq N_0$, the recurrences give $a(n)(S(n+1) - T(n+1)) = 0$. Cancellation gives equality at $n+1$. Induct from N_0 and combine with the separately checked prefix. $\square$

The recorded cubic squared-binomial moment has $a(n) = n^4 - n^3 = n^3(n-1)$ and $N_0 = 2$. It would be wrong to start recurrence cancellation at zero or one. The mutation suite changes the regularity start to zero and confirms rejection. The identity itself is still meaningful at the singular indices; it is the inference of the next value that fails there.

5.6 Selected discovered recurrences

The following table records a, b with the convention $a(n)S(n+1) = b(n)S(n)$. It is an artifact summary, not a claim that these are minimal or uniquely normalized recurrences.

m	$w(k)$	$a(n)$	$b(n)$	N_0
1	1	1	2	0
1	k	n	$2n + 2$	1
1	k^2	n	$2n + 4$	1
1	k^3	$n^3 + 3n^2$	$2n^3 + 12n^2 + 18n + 8$	1
2	1	$n + 1$	$4n + 2$	0
2	k	n^2	$4n^2 + 2n$	1
2	k^2	n^3	$4n^3 + 6n^2 - 2$	1
2	k^3	$n^4 - n^3$	$4n^4 + 6n^3 - 8n^2 - 6n + 4$	2

A common factor of a and b cannot automatically be canceled from the *flux certificate* unless the resulting flux still satisfies the polynomial identity. The producer only cancels a common polynomial factor of a, b, U and then performs full replay. Other simplifications may be valid as sequence reasoning but need a separate argument at roots of the canceled factor.

5.7 Failure modes and the next genuinely useful generalization

The run deliberately includes two unknown cases. The cubic ordinary-binomial moment is unknown with flux degree at most five and becomes provable when the flux degree reaches six. A cubic-power binomial sum is unknown under the configured first-order bounds. Neither result proves that the sum identity is false.

The next extension should be higher-order telescoping, not unbounded guessing of larger first-order polynomials. Search for

$$\sum_{j=0}^r a_j(n) S(n+j) = 0$$

with a flux expressed relative to a common supported hypergeometric term. The replayer must check every local shift identity, the union of shifted support ranges, the boundary contributions, and the nonvanishing of the leading coefficient on the recurrence range. An inhomogeneous boundary term should be retained rather than erased. This higher-order worker is a design direction, not an implemented feature of BFT.

More general factorial ratios, rational summands, and alternating sums require additional source identities and denominator guards. The input schema should grow one semantic family at a time. Importing a general computer-algebra answer and hoping that `field_simp` resolves all exceptional indices would undo the principal benefit of the boundary-safe design.

6 A concrete Lean and Leant integration plan

6.1 Implement the acceptance lemmas before the search adapter

The production development should begin with a small Lean library containing three semantic results: preservation of a simultaneous zero invariant along a fold, preservation of a binary relation along two folds, and the binomial flux theorem. The Noetherian termination proof and the Gröbner algorithm do *not* have to be formalized before ideal certificates can be accepted. They explain discovery; the certificate soundness theorem alone authorizes the proof.

The supplied `lean/BridgeSpec.lean` contains core Lean source specimens for generic fold invariance, relational invariance, and transport. It has no `sorry` or added axioms, but it is *uncompiled*. Its status

is `NOT_RUN`, not “verified Lean”. It is a starting specimen for a pinned build, not a hidden claim that the package already integrates with Lean.

For polynomial certificates, two replay strategies are sensible. The first emits ordinary Lean identities with concrete coefficients and proves each using the existing ring-normalization infrastructure, then applies an ordinary induction theorem. The second ports the sparse checker to Lean, proves a soundness theorem for its interpretation, and checks data by reduction. Start with the first route: it makes translation errors visible as ordinary Lean goals and avoids prematurely committing to a reflected polynomial implementation.

A positive replay must prove the original source-level target, not just a fresh theorem about the serialized polynomial system. The supplied data’s mathematical meaning and the source-to-data bridge are separate acceptance obligations.

6.2 Proposed modules and their exact responsibilities

These paths are proposed new modules. They are not claimed to exist in the reviewed repository.

Proposed module	Responsibility and required output
<code>Forge/Closure/FoldSoundness.lean</code>	Prove generic unary-invariant, binary-relation, and commuting-map fold lemmas. No solver dependency.
<code>Forge/Closure/PolynomialReplay.lean</code>	Interpret the state-coordinate map, construct rational coefficient expressions, prove initial and multiplier identities, apply simultaneous-zero induction.
<code>Forge/Closure/ReifyTransitions.lean</code>	Recognize supported iterations/folds; produce a transition IR <i>and</i> explicit source equation obligations. Never identify terms by printed names.
<code>Forge/Closure/FiniteContract.lean</code>	Check finite relation tables with the entire alphabet, connect tables to exact source-step equations, derive universal endpoint observations.
<code>Forge/Closure/BinomialFlux.lean</code>	Prove the supported sum bridge, including support-zero facts and regularity-based recurrence comparison.
<code>Forge/Closure/WorkerAdapter.lean</code>	Serialize only supported problems, consume certificate data, and request native replay through Forge’s existing verification boundary.
<code>Forge/Closure/Diagnostics.lean</code>	Report generator count, degree ceilings, missing bridge obligations, singular recurrence indices, and concrete counterexample words.

No module in this plan should expose an operation that marks a goal solved merely because the Python checker returned true. Python is an experimental replay oracle; production authority is the completed Lean proof.

6.3 Reifying an iteration without changing its meaning

The initial supported source grammar should be small: a tuple of scalar state components, total polynomial coordinate updates, and a visible finite iteration or left fold. Fixed finite choices become transition labels. The frontend must extract the equations of the exact recursive definition or use a supplied theorem connecting it to a fold. It should not invent a new recurrence based only on the function’s name or a few evaluations.

A concrete extraction procedure is:

1. Introduce the universal input word or iteration count while preserving the local telescope. Locate a supported fold/iterate expression on each side of the target.
2. Form a product state when comparing two computations. Enumerate state coordinates by exact expressions with their types, not by pretty-printed variable names.
3. Reify the target equality as a polynomial difference. For every coordinate update, construct a source equality identifying it with the interpretation of the proposed polynomial.
4. Prove the initial-state equality and every update equality. Arithmetic casts and any finite-case expansion are included here.
5. Send the resulting polynomial problem to PRC, then GI when appropriate. On a positive certificate, prove all concrete multiplier identities in Lean.
6. Apply the simultaneous-zero fold lemma and rewrite through the source equations to close the original target.

A target such as equality of two fold observations should generate *one* product-state relation, rather than independently proving a closed form for each implementation. This often avoids unnecessary expression growth. The certificate may describe a stronger invariant than the target, but that invariant remains local to the proof unless deliberately exported as a separately proved theorem.

Natural and integer source values. A rational polynomial model for natural-number arithmetic needs a proved embedding. Natural subtraction is not ring subtraction; division is not field division; finite-width overflow is not integer addition. The first reifier should therefore admit only operations for which a direct homomorphic interpretation is proved. Unsupported operations remain outside this worker. For finite-width values, finite-state tables may be an appropriate alternative when the state space is genuinely small.

Opaque subexpressions. An opaque expression may be a polynomial atom in a *single* algebraic identity. It cannot automatically become a state coordinate with an invented polynomial update. If a recursive step changes the opaque expression, its new value needs a source equation or a separate invariant. This is the difference between reifying a static polynomial goal and reifying an entire transition semantics.

6.4 Replay through ordinary Lean terms

For a fixed certificate with r generators, build the proposition

$$I(s) := \bigwedge_{i=1}^r b_i(s) = 0.$$

An implementation may use a finite universal quantifier instead of a nested conjunction. Prove $I(s_0)$ by exact normalization. In the step proof, instantiate the supplied multiplier identity for row i and rewrite every $b_j(s)$ to zero using the induction assumptions. The resulting equality is immediate.

The generators need not be linear, and the multipliers need not be nonnegative. A generic theorem should quantify over arbitrary functions b_i and h_{aij} satisfying the displayed equalities. Polynomial normalization is only the way the adapter proves those premises. This keeps the induction library reusable for non-polynomial identities later.

For a small initial implementation, generating a conjunction of perhaps a dozen identities is preferable to building a high-performance matrix reflection layer. Once proof sizes are measured, share common subexpressions, factor coefficient arithmetic, and introduce named intermediate identities inside the proof term. Do not assume that the fewest generators automatically gives the smallest Lean proof; a smaller generator set can require much larger polynomial multipliers.

6.5 Finite-state replay and existing Mathlib infrastructure

Mathlib’s `Mathlib.Computability.DFA` exposes a transition function, start state, and fold-based evaluation, including `DFA.evalFrom`, `DFA.evalFrom_cons`, and related lemmas [14]. Boolean recognition can use that representation directly. General Moore outputs require a thin observation layer rather than pretending that DFA acceptance is already an arbitrary-output equivalence API.

For explicit finite state types, enumerate all constructors with a genuine finite-type instance and prove the transition table entries by reduction. Encode relation membership as a finite decidable predicate and prove the three certificate premises. A strict replay can use ordinary kernel reduction or a proved reflected checker; the package does not require `native_decide` or an unexamined compiled-evaluation shortcut.

The Leant hook belongs *after* exact candidate typing and assertion preflight, and before reporting the behavioral assertion inconclusive. It should activate only when the assertion is recognized as a supported all-words contract and both source bridges are available. The result then proves that exact assertion for that exact candidate. It must not change the ownership or identity of the candidate supplied by Djex.

6.6 Telescoping replay and finite sums

The first Lean BFT module should fix the summand family (16) and accept only the polynomial certificate (18). Reification must verify that the input sum has the exact range, weight, binomial power, and casts required by the family. A syntactically similar rational summand outside the family is not accepted by the same theorem.

The bridge proof should operate with scaled binomial identities over an ordered characteristic-zero field, using the natural indices only for bounds and finite sets. In particular, $n + 1 - k$ in the symbolic identity is a field subtraction after casting; conversion from a natural subtraction is justified by $k \leq n + 1$. Prove the two support-zero endpoint facts before any division simplification. Then use the finite-sum telescoping lemma and the single guard $n + 1 \neq 0$.

To close an equality with a proposed closed form, create separate obligations for that form’s recurrence, the finite prefix, and the leading-coefficient regularity range. These are explicit AND children of the original goal. The flux search may succeed while the closed-form comparison remains open; that is useful partial progress, not a completed sum proof.

6.7 Concrete dependencies, without a speculative dependency stack

Library	Concrete use	Status and constraint
Python standard library	<code>Fraction</code> , sparse dictionaries, JSON, BFS	Used in the independent checker; no SymPy import on replay.
SymPy 1.14.0	<code>Poly</code> , <code>gauss_jordan_solve</code> , <code>nullspace</code> , <code>groebner</code> , membership and reduction	Used by all algebraic producers. Computations are untrusted and checked afterward.
python-flint	<code>fmpq_mat.rref</code> , rank, rational matrix operations	Optional producer optimization, not installed or tested here. Rectangular membership needs RREF; its <code>solve</code> interface assumes square invertible input [13].
Mathlib	Polynomial interpretation, ring normalization, finite sums, <code>Nat.choose</code> , DFA fold lemmas	Proposed Lean replay foundation. Only selected source/API reads were inspected; no package compatibility build was run.

SymPy’s official polynomial interface documents exact polynomial objects and Gröbner membership/reduction operations [12]; the executed producer code is the most precise record of which APIs were actually used. FLINT should remain interchangeable with SymPy behind the producer interface. Neither belongs in the trusted acceptance boundary.

A large collection of optional SMT, SAT, e-graph, or external-prover dependencies would not improve these particular workers. Forge already discusses such integrations. The initial implementation can be a small exact algebra producer plus ordinary Lean replay, and a standard-library finite-state producer. This keeps the first end-to-end proof achievable and the failure surface understandable.

6.8 Selecting a worker from a proof obstruction

A deterministic selector can be based on recognizable structure rather than a learned ranking model. A supported finite-state fold contract goes to FRC. A polynomial iteration equality goes first to PRC when degree preservation or small observed growth makes finite-dimensional closure plausible. If pullbacks show increasing degree or repeated polynomial divisibility structure, route to GI. A supported binomial-power sum goes to BFT. Unrecognized structure falls back to the existing Forge plan.

Do not treat the worker’s failure as a rejection of the theorem. A vector-space ceiling can trigger the ideal worker; an ideal multiplier ceiling can trigger a larger reconstruction budget without rerunning discovery; a telescope can export its recurrence while leaving comparison obligations open. The new diagnostic value lies in naming the concrete obstruction: “new independent pullback of degree 16”, “ideal membership found but multiplier degree cap reached”, “finite product relation not exhausted”, or “leading recurrence coefficient unresolved at index 1”.

The selector itself can initially be small enough to audit exhaustively. Optimizing it statistically should wait until there are measurements of complete source extraction and Lean replay, rather than treating the current Python component times as a model of end-to-end tactic cost.

7 Executed experiments and what they establish

7.1 The recorded run

The experiments were executed with Python 3.13.5 and SymPy 1.14.0 on the Linux environment recorded in `results/run.json`. The deterministic corpus generator uses seed 20260915. All mathematical coefficients are exact rationals. There is no floating-point numerical solver in this package; floating-point numbers occur only in elapsed-time measurements.

There are 317 producer cases. Of these, 178 return positive certificates, 135 return concrete counterexample certificates, and four return UNKNOWN. Thus 313 certificate bundles are stored. They were all replayed in a second invocation using Python’s `-S` option, which disables site-package initialization. The checker imports no search module and no SymPy module.

Producer	Cases	Positive	Counterexample	Unknown
PRC: vector closure	44	26	17	1
GI: ideal closure	19	18	1	0
FRC: finite contracts	242	124	117	1
BFT: binomial flux	12	10	0	2
Total in this one run	317	178	135	4

The total is a count of cases in this one explicitly generated corpus. It is not added to any of Forge’s existing proposal runs. It is also not a “solved Lean goals” count: the corpus starts from mathematical problem data, before any source-to-IR extraction or Lean replay.

7.2 What was generated

The vector corpus contains two named positive examples, 24 generated synchronized affine pairs, 17 false recurrence assertions, and the repeated-squaring degree-ceiling control. The affine pairs have two arbitrary integer-coefficient affine updates on a two-component state and a synchronized copy, with a randomly chosen quadratic observation difference. They are intentionally structured positives rather than uniformly sampled arbitrary polynomial claims.

The ideal corpus contains repeated squaring, a joint-generator product example, 16 synchronized nonlinear scalar pairs with two update symbols, and the word-order counterexample. Generator discovery and multiplier reconstruction are both exercised. It is not a large Gröbner stress test: the examples have low dimension and small coefficients, and most nonlinear positives admit one obvious algebraic relation even though the multiplier varies.

The main finite-state corpus has 240 cases. Half are constructed equivalent by relabeling a machine and adding an unreachable junk state; half are independently generated pairs. State counts of the original machines range from two through eight, with alphabet sizes one through three. The generated pairs are compared with an independent greatest-fixed-point procedure over the full product relation. It repeatedly deletes output-disagreeing or successor-inconsistent pairs, rather than using the producer’s forward BFS. All 240 outcomes agree: 124 equivalent pairs and 116 inequivalent pairs.

The finite-state controls add the length-17 distinguishing-word example and its pair-budget truncation. The telescoping corpus contains ten positive moment families: powers $m = 1, 2$ with weights $1, k, k^2, k^3$, plus $2k^2 + 3k + 1$ for both powers. The two unknown controls are the lower-degree cubic-moment search and the capped cubic-power first-order search.

7.3 The meaningful ablations

Three small comparisons illustrate different gains without claiming a comparison against Lean tactics.

Conservation versus coupled invariance. For the two named vector examples, the target polynomial is not conserved. The supplied matrix identities prove preservation of the joint zero condition. This separates the proposed certificate language from the narrower test $I \circ F = I$; it does not prove that another existing invariant-search strategy could not find the same result.

Vector closure versus ideal closure. Repeated squaring produces independent pullbacks of degrees $1, 2, 4, 8, 16, \dots$. PRC returns UNKNOWN at its degree limit. GI returns one generator and the multiplier $x + y$. This is supported both by an exact algebraic obstruction and by the recorded runs, rather than merely by a timing difference.

Finite samples versus an all-words contract. All nine sampled word lengths from zero through eight agree for the delayed-difference machine, but its shortest distinguishing word has length 17. A complete product exploration finds it. An eight-pair truncated exploration remains UNKNOWN. This directly tests that neither sampling nor a resource stop is mistaken for universal evidence.

7.4 Mutation tests and independent arithmetic tests

All 25 targeted invalid mutations are rejected. These include a wrong vector coefficient, a missing transition, a changed initial state, a changed target, a floating or Boolean coefficient, a zero denominator, a noncanonical rational, a removed ideal generator, a wrong polynomial multiplier, a reversed execution word, missing or duplicate relation entries, changed observations, an invalid input symbol, an altered flux, an altered summand, an unsupported binomial power, and a false recurrence-regularity start.

These are *targeted invalid* mutations. It would be mathematically wrong to demand rejection of every conceivable edit: a redundant invariant, a different valid relation, or a changed informational origin annotation may still constitute a sound certificate. One unit test deliberately changes the origin annotations while preserving all proof identities, and acceptance remains valid. The checker must verify mathematics, not that a certificate was generated by a particular run.

The separate unit suite has 12 test methods. One method replays all 313 stored bundles. Another performs 200 deterministic random rational-polynomial cases, comparing the independent checker's addition, multiplication, and simultaneous substitution against SymPy: 600 operation comparisons. Other methods check empty-word counterexamples, a zero target requiring an empty basis, exact parsing, the distinction between generator and multiplier ceilings, and fixed-generator reconstruction. These test counts are different units and are not combined into an inflated "number of proofs".

7.5 Symbolic certificates versus numerical diagnostics

Each of the ten moment recurrences is also evaluated directly at $n = 0, \dots, 20$, for 210 diagnostic comparisons using exact binomial integers. This checks the connection between the experiment's input convention and the derived recurrence on a small range. It is not the basis of acceptance: a corrupted flux is rejected by the polynomial identity checker regardless of whether a finite sample happens to agree.

The stronger evidence is the exact identity plus the fully stated mathematical bridge in Section 5. The remaining formalization task is to express and prove that bridge in Lean and bind it to the original sum expression. The diagnostic table must not be cited as a substitute for that task.

7.6 Resource settings and one retained failed development run

Default algebraic discovery ceilings are 32 retained generators, polynomial degree 12, 10,000 terms per processed polynomial, and 1,000 generated pullbacks. Ideal multiplier reconstruction is capped at degree four. The ordinary finite-state pair ceiling is 10,000. Positive telescoping experiments use coefficient degree at most six and total flux degree at most nine; the negative controls deliberately use smaller bounds.

An initial experiment incorrectly expected the ordinary cubic-binomial moment to fit a total flux degree bound of five. It returned UNKNOWN. A larger run found a degree-six flux. The original failure is retained in `results/initial-cap-failure.txt`; the lower-cap case is now an expected unknown regression. No identity check was weakened to make the experiment pass. This is useful evidence about the required ansatz size, not a failure of mathematical correctness.

The checker has its own representation and intermediate-arithmetic ceilings. A search may theoretically find a certificate that those ceilings refuse. Production should report this as a resource or replay outcome, not silently raise a trust ceiling. The supplied functions are research interfaces and do not yet normalize every possible resource exception into a production-grade result type.

7.7 Timing and its limitations

The CSV contains per-case producer and replay durations. Producer timers include construction of the search object, coefficient solving or traversal, certificate construction, and the producer’s final Python replay. A second timer measures another checker invocation on the in-memory data. Timers exclude Python startup, SymPy import, original problem construction, JSON file writing, source reification, Lean elaboration, and Lean kernel checking.

In the recorded run, the 24 generated vector cases take about 4.24 seconds in total inside the producer timers; the 16 generated ideal cases about 0.11 seconds; and the ten positive telescoping cases about 3.02 seconds. The 240 small finite-state cases together take only a few milliseconds and are below a useful granularity for detailed performance inference in this environment. Exact values remain in the CSV and JSON rather than being rounded into a speedup claim.

These are one-run component observations, not stable latency estimates. The families differ in structure and difficulty; their times cannot rank the general usefulness of the workers. Most importantly, *no experiment invokes `grind`, `Aesop`, `nlinarith`, or any other Lean tactic*. There is no measured end-to-end solved-goal gain, no Lean proof-size measurement, and no compiled trust-profile comparison.

7.8 What the evidence does and does not support

The evidence supports the existence of working certificate discovery algorithms for the stated fragments, the successful replay of their outputs by a separately implemented arithmetic checker, several sharp distinctions between the certificate languages, and rejection of specified malformed or false inputs. It also supports the feasibility of avoiding a trusted Gröbner backend in the positive acceptance path.

It does not establish that the Python checker is formally verified, that the JSON boundary is hardened against hostile resource exhaustion, that the source-to-IR bridges are correct, or that the proposed

Lean tactic solves more real-world goals under a fixed budget. Those remaining tasks are explicit development gates rather than hidden assumptions.

8 Implementation priorities and acceptance gates

8.1 First milestone: one complete polynomial proof path

Implement the fold-soundness lemmas and ordinary-term replay for one small PRC certificate, then connect a single explicit Lean recurrence to the transition IR. The acceptance case should be the sign-changing invariant, with both the positive theorem and a changed-update negative control. Do not begin by porting the entire producer to Lean or by changing Forge’s scheduler.

The gate is concrete: from the exact original Lean source, generate the problem, run the producer, reconstruct the proof, and kernel-check the final declaration under the project’s approved trust profile. The same test must fail safely when the transition, initial state, or target is changed without updating the certificate. Archive the exact source and final proof status. This turns a mathematical certificate into an actual automation feature.

8.2 Second milestone: the nonlinear ideal path

Reuse the same zero-invariant induction theorem and extend only the identity replay to polynomial multipliers. Port the repeated-squaring and joint-generator examples. A successful test must show that the Gröbner engine is absent from the acceptance path: replay a stored certificate in a process or build environment without the search library.

Keep discovery and multiplier reconstruction separate. When reconstruction times out, retain the frozen generator list as untrusted reusable input, not as a proved invariant. Instrument generator count, degree growth, multiplier degree, coefficient bit size, and final Lean expression size. Only then compare direct normalizer proofs against a reflected sparse checker.

8.3 Third milestone: a universal behavioral assertion

Add FRC to Leant for an explicit finite-state fold, first with source states already represented by a finite inductive type. Prove the source-to-table bridge, not just the table equivalence. The acceptance case should quantify over *all* finite input words. Include the delayed mismatch, changed output table, and truncated product exploration as controls.

This milestone must not be declared complete by replaying a fixed finite collection of words. Nor should it be substituted with a proof about the candidate’s type alone. The useful deliverable is an exact universal behavioral assertion about the candidate that Leant actually displays.

8.4 Fourth milestone: summation with checked boundaries

Formalize the binomial flux theorem for the restricted family and prove the central-binomial identity by recurrence comparison. Add the singular-index moment to force explicit treatment of $N_0 > 0$. The gate should reject a recurrence-comparison proof that cancels $a(0) = 0$, even if the recurrence certificate itself is correct.

After that, extend the supported summand family or recurrence order. A generic hypergeometric importer is premature before the support, casting, boundary, and regularity obligations are independently

tested. The payoff of BFT is not merely another closed form; it is a reusable proof route for an entire class of parameterized finite sums.

8.5 The first fair Lean evaluation

A future Lean corpus should include held-out recurrences, product-state equivalences with different state representations, nonlinear invariants with nontrivial initial assumptions, universal finite-fold contracts, and binomial sums not present in the development examples. Preserve false variants and resource-limited variants. Split structurally similar families across train/development and evaluation sets rather than shuffling only coefficient values.

Compare the same source files under the same Lean and Mathlib revisions, heartbeat and wall-clock budgets, import costs, and trust profile. Count only completed original goals. Report discovery time, reification time, replay time, proof size, and failures separately. A cached certificate replay is a useful workflow but not the same experiment as fresh proof discovery.

The strongest near-term hypothesis is not that Forge universally dominates `grind`. It is that these workers eliminate specific human-supplied objects: an auxiliary invariant, a bisimulation relation, or a telescoping flux. A fair evaluation should test that hypothesis directly.

8.6 Research directions that remain genuinely open in this design

Three directions are particularly valuable. First, coefficientwise closure for arbitrary polynomial input values would move beyond finite input alphabets and address infinite-domain list folds. Second, control-state-indexed ideals would combine finite control and algebraic data without forcing either representation to model the other poorly. Third, higher-order telescopers would cover sums outside the first-order schema while preserving explicit boundary obligations.

Each should keep the current asymmetry: an ambitious producer may use sophisticated mathematics, but its acceptance object should remain small and explicit. Noetherianity can guide a search without becoming a trusted runtime claim; a shortest counterexample can guide synthesis without becoming a non-inhabitation theorem; a recurrence can guide a sum proof without silently erasing exceptional indices.

9 Conclusion

The practical step beyond Forge’s reviewed design is to let a goal generate the finite structure needed for its own induction. PRC closes a vector of observations under pullback. GI closes the ideal generated by those observations and thereby handles nonlinear relations that have no finite orbit span. FRC turns a finite relation into an all-words behavioral proof. BFT turns a local polynomial flux identity into a recurrence for a sum whose support moves with its parameter.

The package implements all four discovery routes and checks their outputs independently in Python. The remaining work is not mysterious, but it is substantial: source reification, semantic bridges, Lean replay, and an honest end-to-end evaluation. The report supplies the proof obligations and acceptance gates for that work rather than describing an unimplemented tactic as already complete.

A Artifact inventory and reproduction

A.1 Files in the archive

Forge_Closure_Extensions/	
article/forge_closure.tex	main LaTeX source
article/forge_closure.pdf	rendered article
article/sections/*.tex	article sections
article/references.tex	ordinary LaTeX bibliography
prototype/forge_closure/	
search.py	four untrusted producers
checker.py	independent exact checker
verify.py	checker-only command line
prototype/run_experiments.py	deterministic corpus and mutations
prototype/tests/test_closure.py	unit and arithmetic tests
prototype/requirements-search.txt	
results/run.json	environment, configuration, outcomes
results/cases.csv	individual case records
results/certificates/*.json	313 replayable problem/certificate pairs
results/diagnostics.json	discovered recurrences and sample control
results/mutations.json	25 targeted-invalid results
results/unit-tests.txt	executed unit-test log
results/stdlib-replay.txt	checker-only replay log
results/initial-cap-failure.txt	retained development failure
lean/BridgeSpec.lean	uncompiled generic bridge specimen
audit/review.json	exact review scope and source identities
README.md	entry points and status
build_article.py	reproducible LaTeX build command

The code is original package code. No upstream repository checkout, third-party paper PDF, font file, or external search dependency is redistributed. The package does not install itself into the user's Forge repository and does not modify any remote repository.

A.2 Replaying certificates without the producer dependencies

From the package's prototype directory, run

```
python -S -m forge_closure.verify ../results/certificates
```

The command accepts a directory and enumerates its JSON files, so it does not depend on a shell expanding a wildcard. It validates the stored mathematical problem and certificate pair. For a production tactic the problem must instead come from the trusted source reifier; receiving both fields from an external producer would let that producer change the question.

The checker returns success only for a supported certificate kind. Malformed exact numbers, missing dimensions, invalid indices, wrong identities, and configured representation ceilings raise a rejection. This is a research parser with several defensive limits, not a promise of robustness against every maliciously expensive input.

A.3 Reproducing discovery and the tests

Install the search dependency, then run the generator and unit tests:

```
cd prototype
python -m pip install -r requirements-search.txt
python run_experiments.py --out reproduced-results
python -m unittest discover -s tests -v
python -S -m forge_closure.verify reproduced-results/certificates
```

The generator defaults to a new **reproduced-results** directory rather than the recorded evidence directory. Re-running it preserves the seed and mathematical construction, but timing values may differ. The unit test replay targets the shipped corpus and also runs independently generated arithmetic cases. The package was tested on Python 3.13.5. It uses modern standard-library features such as strict **zip**; older Python releases are not a tested compatibility target. SymPy is pinned to the executed version. There are no optional search dependencies required to replay certificates.

A.4 Building the article

With pdfLaTeX available, run from the package root:

```
python build_article.py
```

The script performs three pdfLaTeX passes. It uses stock LaTeX packages and Latin Modern fonts supplied through TeX, with no shell escape, online service, or separately distributed font file. The editable source is split into sections to make later changes manageable.

B Wire format and checking obligations

B.1 Exact arithmetic

A rational number is a pair of literal integers **[numerator, denominator]**, with positive denominator and reduced form. Booleans and floating-point numbers are rejected even though some host-language conversions would treat them as integers. A sparse polynomial is a list of pairs consisting of an exponent vector and an exact rational coefficient. Duplicate monomials and zero coefficient entries are rejected; the empty list represents zero.

For instance, in variable order (x, y) ,

```
[[[1,0],[1,1]], [[0,1],[-1,1]]]
```

represents $x - y$. The complete variable ordering is part of the problem schema. The source frontend must preserve the interpretation of that ordering; changing the order without changing the interpretation changes the theorem.

The checker bounds dimensions, term counts, exponents, coefficient bit lengths, basis sizes, and finite-state counts. Some intermediate products are materialized before all resource checks complete. A production service therefore still requires process-level resource isolation and a validated input-size policy.

B.2 Polynomial closure bundles

The problem kind is `polynomial_transition_system_v1`. It contains dimension, rational initial state, the full list of polynomial coordinate updates for each transition, and every target polynomial. The positive kinds are `pullback_closure_v1` and `pullback_ideal_v1`. Both contain a basis, a step matrix for every original transition, and target reconstruction rows. Matrix entries are rationals in PRC and polynomials in GI.

The informational `origins` entries help explain discovery, but positive replay does not trust them. The negative kind `transition_word_counterexample_v1` instead contains a target index and an execution word. Its checker executes the original transition system exactly and tests the indicated target at the final state. It does not accept a producer-supplied final state as evidence.

B.3 Finite-state and telescoping bundles

The finite-state problem contains both complete machines, their initial indices, output tables, and a common alphabet size. A positive certificate lists related state pairs; a negative one lists a word. Every transition symbol is checked, including symbols not encountered in a sample suite.

The summation problem kind is `binomial_moment_v1`, carrying m and w . The certificate kind `binomial_flux_v1` carries a, b, U, N_0, σ . Replay reconstructs (18) and (23). It does not accept a precomputed zero residual, a numerical table of the sum, or a stored claim that the recurrence was checked elsewhere.

C Review ledger and provenance limits

The audit file records Forge’s README, `article/sections/05-structural.tex`, selected ranges of `06-nonlinear.tex` and `07-boolean-and-external.tex`, and `docs/IDEAS-FROM-LEANT-DJEX.md`. It also records the neighboring READMEs, Leant’s `docs/behavioral-synthesis.md`, and the pinned Mathlib binomial source. Some connector responses were truncated; those are explicitly marked as partial reads.

The bibliography points to pinned repository revisions where available, and to official library documentation or primary research sources for outside facts. The non-duplication comparison is against the reviewed merged design at the recorded snapshot, not a guarantee that no related sentence appears anywhere in all historical submissions. The code and experiments were developed for this report and do not claim inherited acceptance from any prior Forge, Leant, or Djex run.

References

- [1] Vladimir Reshetnikov et al. *Forge: README*. Repository snapshot 674521027d968d59f7b83220ed52304a85cb55e2, inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/README.md>.
- [2] Forge contributors. *Structural reasoning section of the merged article*. Merged article, `article/sections/05-structural.tex`, same Forge snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/05-structural.tex>.
- [3] Forge contributors. *Nonlinear arithmetic: exact certificates, not floating-point proofs*. Merged article, `article/sections/06-nonlinear.tex`, same Forge snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/06-nonlinear.tex>.
- [4] Forge contributors. *Boolean search, equality exploration, and external engines*. Merged article, `article/sections/07-boolean-and-external.tex`, same Forge snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/07-boolean-and-external.tex>.
- [5] Forge contributors. *Ideas from Leant and Djex*. `docs/IDEAS-FROM-LEANT-DJEX.md`, same Forge snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [6] Vladimir Reshetnikov et al. *Leant: a Djex-based synthesis REPL for Lean 4*. README, observed snapshot 6bf05ad78c467989e68290f2d08bbbed40802d485. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/README.md>.
- [7] Leant contributors. *Behavioral assertions during synthesis*. `docs/behavioral-synthesis.md`, same observed Leant snapshot. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/behavioral-synthesis.md>.
- [8] Vladimir Reshetnikov et al. *Djex*. README, observed snapshot e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/README.md>.
- [9] Markus Müller-Olm and Helmut Seidl. *Computing polynomial program invariants*. Information Processing Letters 91(5), 233–244, 2004. DOI: 10.1016/j.ipl.2004.05.004. Author preprint inspected: <https://www.uni-muenster.de/imperia/md/content/informatik/agmueller-olm/publications/ipl-poly-preprint.pdf>.
- [10] Marco Almeida, Nelma Moreira, and Rogério Reis. *Testing the equivalence of regular languages*. EPTCS 3, 47–57, 2009. DOI: 10.4204/EPTCS.3.4. <https://arxiv.org/abs/0907.5058>.
- [11] Christoph Koutschan. *Creative telescoping for holonomic functions*. In C. Schneider and J. Blümlein, eds., *Computer Algebra in Quantum Field Theory: Integration, Summation and Special Functions*, Springer, 2013. DOI: 10.1007/978-3-7091-1616-6_7. <https://arxiv.org/abs/1307.4554>.
- [12] SymPy developers. *Polynomials Manipulation Module Reference*. Official SymPy 1.14.0 documentation, inspected 15 September 2026. <https://docs.sympy.org/latest/modules/polys/reference.html>.

- [13] python-flint developers. *fmpq_mat: matrices over rational numbers*. Official documentation, inspected 15 September 2026. Optional producer dependency only. https://python-flint.readthedocs.io/en/latest/fmpq_mat.html.
- [14] Mathlib contributors. *Mathlib.Computability.DFA*. Official generated API documentation, inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Computability/DFA.html.
- [15] Mathlib contributors. *Mathlib/Data/Nat/Choose/Basic.lean*. Source inspected at Forge's stated Mathlib revision 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa. <https://github.com/leanprover-community/mathlib4/blob/1cf325a0cf67aca2b04d76b5380ff6a9e410aefa/Mathlib/Data/Nat/Choose/Basic.lean>.